



Universidad de Valladolid

Escuela de Ingeniería Informática

TRABAJO FIN DE GRADO

Grado en Ingeniería Informática
Mención en Ingeniería de Software

**Aplicación móvil para la información y
geolocalización de procesiones y eventos
en semana santa.**

Autor:

Elena Martínez Vázquez

Dedicatoria

Agradecimientos

Resumen

Abstract

Índice general

1. Introducción	1
1.1. Contexto	1
1.2. Motivación	2
1.3. Estado de la cuestión	2
1.3.1. Aplicaciones actuales de Semana Santa	3
1.3.2. Aplicaciones de seguimiento en tiempo real	5
1.4. Objetivos	6
1.5. Estructura de la memoria	7
2. Planificación	9
2.1. Metodología	9
2.2. Planificación, Fases e iteraciones	9
2.2.1. Planificación inicial	9
2.2.2. Iteración 1: Módulo ciudadano básico	11
2.2.3. Iteración 2: Sistema de mapas	12
2.2.4. Iteración 3: Geolocalización y perfil cofrade	12
2.2.5. Iteración 4: Panel de la Junta de Cofradías	12
2.2.6. Iteración 5: Panel de administrador y notificaciones	13
2.2.7. Fase final: documentación y entrega	13
2.3. Análisis de riesgos	13
2.3.1. Planificación de riesgo	14
2.3.2. Monitorización de riesgos	14
2.4. Seguimiento del proyecto	16
2.5. Estimación de costes	16
2.5.1. Costes de personal	16
2.5.2. Costes de equipo y amortización	17
2.5.3. Costes de licencias de software	18
2.5.4. Costes estructurales	18
2.5.5. Coste total	18
3. Análisis	21
3.1. Requisitos	21
3.1.1. Requisitos funcionales	21
3.1.2. Requisitos de información	23

3.1.3. Requisitos no funcionales	24
3.2. Casos de uso	25
3.3. Modelo de dominio	26
4. Descripción de las iteraciones	33
4.1. 1ª Iteración	33
4.1.1. Análisis	33
4.1.2. Diseño	33
4.1.3. Implementacion	40
4.1.4. Pruebas	40
4.2. 2ª Iteración	40
4.3. 3ª Iteración	40
4.4. 4ª Iteración	40
5. Estado final de la aplicación	41
5.0.1. Historias de usuario	41
6. Pruebas	43
7. Conclusiones	45
7.1. Trabajo futuro	45
Apéndices	49
A. Acrónimos	49
B. Manual de despliegue	51
C. Manual de uso	53
D. Contenido del TFG	55
Bibliografía	59

Índice de figuras

1.1. Interfaz de la aplicación El Penitente [?]	3
1.2. Interfaz de la aplicación Valladolid Cofrade [?]	4
1.3. Ejemplo de geolocalización en tiempo real en Google Maps	5
1.4. Seguimiento en tiempo real de un vehículo en Uber	6
2.1. Diagrama de Gantt general completo del proyecto	11
3.1. Diagrama de caso de uso Ciudadano	25
3.2. Diagrama de caso de uso Cofrade	25
3.3. Diagrama de caso de uso Junta de Cofradías	26
3.4. Diagrama de caso de uso Administrador	26
3.5. Diagrama de dominio del proyecto	32
4.1. Diagrama de paquetes de la aplicación cliente	36
4.2. Diagrama de despliegue de la aplicación	37
4.3. Diagrama de dominio	39

Índice de tablas

2.1. Riesgos identificados en el proyecto	14
2.2. Riesgo 1: Estimaciones de tiempo erróneas	14
2.3. Riesgo 2: Problemas con la integración de Google Maps	14
2.4. Riesgo 3: Errores durante el desarrollo	15
2.5. Riesgo 4: Falta de experiencia con tecnologías móviles	15
2.6. Riesgo 5: Problemas con la geolocalización en tiempo real	15
2.7. Riesgo 6: Cambios en los requisitos	15
2.8. Riesgo 7: Especificaciones genéricas	15
2.9. Riesgo 8: Problemas con el entorno de desarrollo	16
2.10. Riesgo 9: Indisposición del desarrollador	16
2.11. Riesgo 10: Problemas con Firebase/base de datos	16
2.12. Reparto de horas y coste de personal por perfil	17
2.13. Amortización total de los recursos de equipo	18
2.14. Coste de licencias de software	18
2.15. Costes totales del proyecto	19
3.1. Caso de uso: Visualizar procesión en tiempo real	27
3.2. Caso de uso: Compartir ubicación durante procesión	28
3.3. Caso de uso: Crear alerta	29
3.4. Caso de uso: Generar ruta hacia procesión	30
3.5. Caso de uso: Gestionar ciudades disponibles	31
5.1. Historia de usuario HU01	41

Capítulo 1

Introducción

1.1. Contexto

La Semana Santa es una de las fiestas culturales y religiosas más relevantes del calendario español. Detrás de esta tradición hay siglos de historia, ya que forma parte del patrimonio histórico y artístico del país. En ciudades como Sevilla, Málaga o Valladolid, entre otras, está declarada Fiesta de Interés Turístico Internacional. Esta celebración tiene un gran impacto tanto cultural como económico, debido a los miles de visitantes nacionales e internacionales que atrae cada año.

La estructura organizativa de la Semana Santa está formada por las cofradías o hermandades, las cuales son asociaciones religiosas encargadas de organizar procesiones y eventos durante estos días. Cada procesión sigue su itinerario por las calles de la ciudad, llevando consigo pasos de gran valor artístico que representan escenas de la Biblia. Esta organización de horarios, recorridos y actos requiere una gran coordinación por parte de las distintas cofradías y de la Junta de Cofradías de cada ciudad. Durante estos días, la ciudad experimenta imprevistos, como cortes de tráfico, cambios en la movilidad urbana y una gran cantidad de personas en puntos estratégicos del recorrido.

Desde el punto de vista social, la Semana Santa puede implicar dificultades de movilidad debido al desconocimiento de la ubicación de las distintas procesiones, además de los cambios de última hora en cuanto a los horarios de comienzo de cada procesión debido al mal tiempo o incluso a cancelaciones. Como esto se sale de la planificación inicial, los cofrades encuentran dificultades a la hora de comunicar estos cambios y decisiones de última hora a los espectadores y fieles. A día de hoy, son las distintas Juntas de Cofradías las que deben gestionar toda esta información de forma manual o con herramientas anticuadas.

En el contexto tecnológico actual, la mayoría de los turistas y ciudadanos hacen uso de sus teléfonos móviles como principal medio para consultar información. Aplicaciones como Google Maps [?] permiten conocer en tiempo real la ubicación (geolocalización [?]) de calles, monumentos, bares o incluso el estado del tráfico o del transporte público. Esta funcionalidad podría aplicarse a la Semana Santa, de forma que facilitaría a los visitantes y ciudadanos información constante y rápida sobre dónde se encuentra una procesión o si ha habido algún cambio o imprevisto en la misma.

Finalmente, debido a la cantidad de dinero que genera el turismo durante la Semana Santa, podemos concluir que es de gran interés para las ciudades que la experiencia del turista y del ciudadano sea positiva y que existan herramientas para que esto se haga realidad, de forma que una aplicación que mejore dicha experiencia repercutirá positivamente.

1.2. Motivación

La principal motivación de este proyecto surge de la dificultad que tienen tanto los ciudadanos como los visitantes a la hora de buscar y obtener información clara y actualizada durante los días de la Semana Santa. Aunque a día de hoy existan programas oficiales, carteles informativos o cuentas oficiales en redes sociales, estos medios no siempre consiguen dar a conocer en tiempo real la situación de las procesiones, ya sea por retrasos o cambios de última hora debidos a factores meteorológicos o a problemas dentro de la propia cofradía.

Debido a esto, los asistentes, en muchas ocasiones, deben desplazarse a los distintos puntos del recorrido sin tener la certeza de que la procesión no haya pasado ya, cuánto tiempo de espera se estima hasta que pase por ese punto o incluso si llegará a pasar debido a cancelaciones. Esto genera desorganización y aglomeraciones innecesarias, además de una experiencia menos satisfactoria para los observadores.

Gracias a estas observaciones, se ve clara la necesidad de contar con un medio común capaz de organizar y difundir la información necesaria para que la experiencia sea lo más clara y accesible posible para cualquier persona.

Como respuesta a estas limitaciones, se plantea la idea de una aplicación móvil que permita la centralización de la información sobre el estado de las procesiones de forma actualizada, además de incluir otras funciones como la visualización de los recorridos y la geolocalización en tiempo real de las mismas. De esta forma, se mejora no solo la experiencia de los usuarios, sino que también se facilita la gestión de la información durante la Semana Santa.

Además, el desarrollo de esta aplicación permite no solo aplicar los conocimientos adquiridos durante el grado sobre análisis de requisitos, diseño de software, desarrollo de aplicaciones móviles y gestión de proyectos, sino también ampliarlos mediante su aplicación en un proyecto real, enfrentándose a problemas prácticos como la integración de distintas tecnologías, el manejo de la geolocalización en tiempo real o la gestión de múltiples tipos de usuarios dentro de un mismo sistema.

1.3. Estado de la cuestión

En la actualidad existen diversas soluciones tecnológicas relacionadas con la Semana Santa, aunque la mayoría de ellas se encuentran limitadas a ámbitos geográficos concretos o se centran únicamente en la difusión de información estática. A continuación, se analizan las principales herramientas disponibles, así como sus características más relevantes.

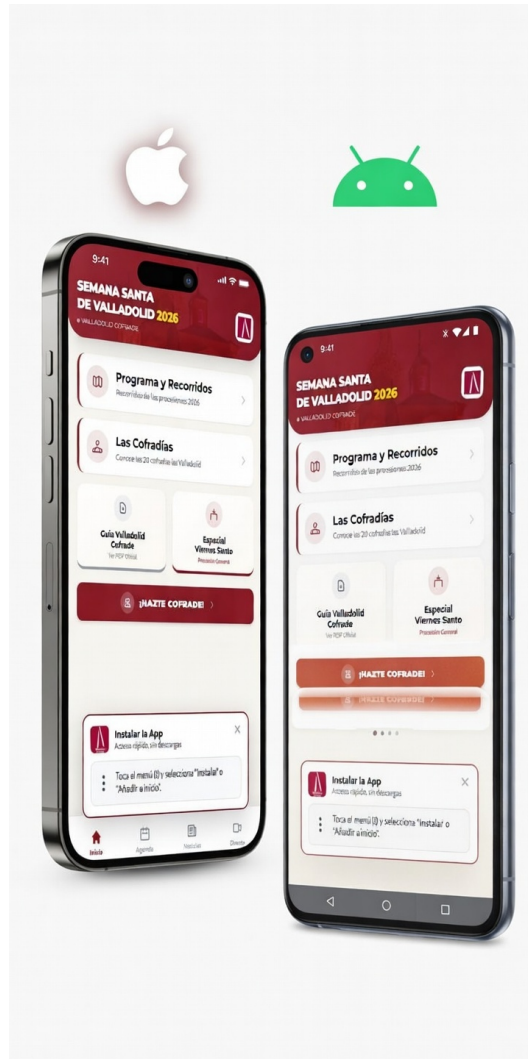


Figura 1.2: Interfaz de la aplicación Valladolid Cofrade [?]

Estas diferencias hacen que cada aplicación tenga un diseño distinto, pensado específicamente para una ciudad concreta, dificultando la creación de una solución reutilizable. Por tanto, en la actualidad no existe una plataforma que consiga unificar y generalizar estos conceptos y ofrecer una visión global de la Semana Santa.

Además de las aplicaciones, los usuarios también hacen uso de las publicaciones oficiales de las cofradías en sus páginas web o de programas impresos. Sin embargo, los medios que realmente resultan útiles para la actualización inmediata de la información son las redes sociales oficiales de las cofradías o de las Juntas de Cofradías, que permiten a los espectadores recibir información sobre cambios o comunicados al instante. Aun así, esta forma de comunicación no siempre resulta fácil de encontrar o de organizar debido a la diversidad de cuentas que gestionan dicha información.

1.3.2. Aplicaciones de seguimiento en tiempo real

En cuanto a la geolocalización y el seguimiento en tiempo real, se trata de una tecnología ya existente que se utiliza en diversas aplicaciones como *Google Maps* [?], que permite conocer la ubicación, cómo llegar a distintos destinos o el estado del transporte público (Figura 1.3).

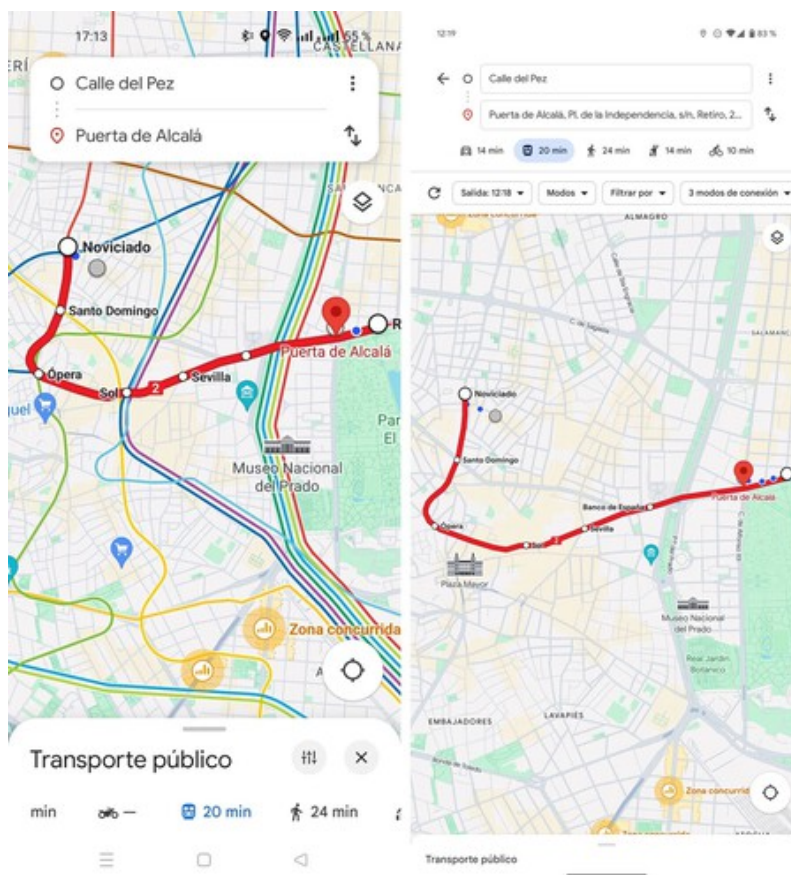


Figura 1.3: Ejemplo de geolocalización en tiempo real en Google Maps

Además, plataformas como *Uber* [?] permiten a los usuarios conocer en todo momento la ubicación exacta del vehículo solicitado (Figura 1.4).

Además, existen diversos Trabajos de Fin de Grado centrados en el desarrollo de aplicaciones móviles y el uso de tecnologías de geolocalización. Un ejemplo es el proyecto realizado por Mario Llorente Pérez [?], enfocado en una aplicación móvil relacionada con actividades deportivas y el uso de mapas y geolocalización. En trabajos como este puede observarse el importante papel que juega la localización en tiempo real y la necesidad de diseñar interfaces que permitan al usuario interactuar con la aplicación de forma sencilla.

Todas estas aplicaciones y trabajos constituyen ejemplos claros de que el seguimiento en tiempo real es una funcionalidad ampliamente utilizada, a pesar de que en el contexto de la Semana Santa sigue estando limitada y, cuando se ofrece, suele restringirse a entornos locales o presentar un nivel de precisión reducido.

Por tanto, su incorporación en una solución a nivel nacional conseguiría no solo mejorar la experiencia

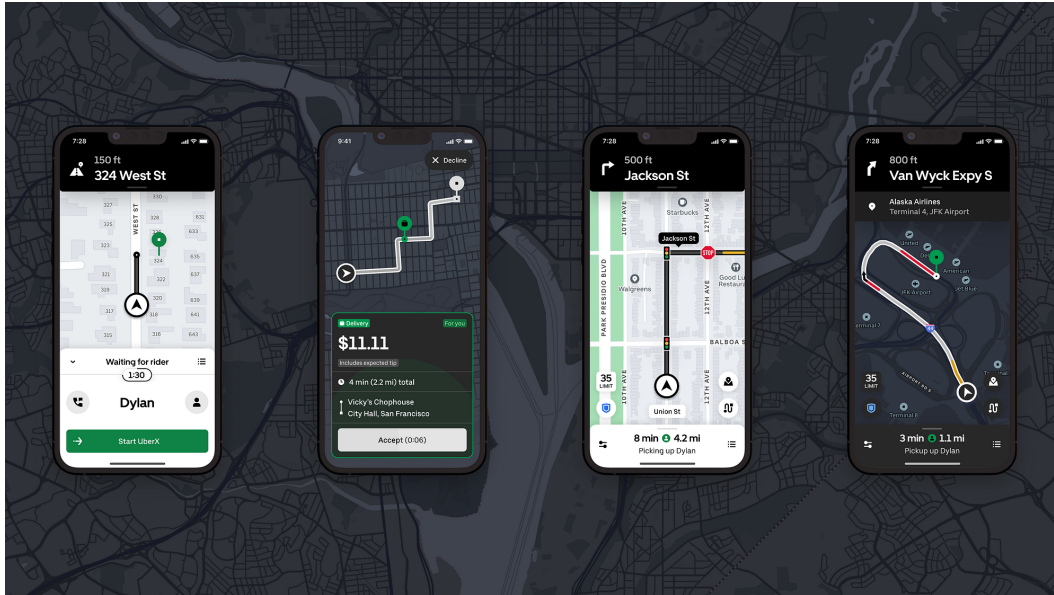


Figura 1.4: Seguimiento en tiempo real de un vehículo en Uber

de los usuarios, sino también superar las limitaciones de las soluciones actuales en cuanto a la oferta de información precisa sobre el estado y la ubicación de las procesiones o eventos.

1.4. Objetivos

El objetivo principal de este Trabajo de Fin de Grado es el diseño y desarrollo de una aplicación móvil orientada a la Semana Santa, capaz de centralizar, gestionar y ofrecer de forma unificada la información de las distintas celebraciones a nivel nacional.

Por tanto, se pretende desarrollar una solución para los usuarios que les permita consultar la información relevante de la Semana Santa de una ciudad y visualizar en tiempo real la ubicación, el recorrido y el estado de las procesiones utilizando la tecnología de la geolocalización.

Para alcanzar este objetivo, se plantean los siguientes objetivos específicos:

- Diseñar dicha aplicación permitiendo a los usuarios elegir entre las distintas Semanas Santas dentro del territorio nacional.
- Proporcionar acceso a la información general relacionada con la Semana Santa, incluyendo historia, cofradías o hermandades y elementos representativos.
- Mostrar información detallada sobre los eventos, como nombre, descripción, fechas, horas e itinerarios.
- Añadir funcionalidades de personalización, como la selección de eventos o elementos de interés y la recepción de notificaciones ante cambios o incidencias.

- Implementar un sistema de visualización sobre mapa que permita representar las procesiones y su evolución en tiempo real.
- Facilitar la navegación del usuario mediante la posibilidad de calcular rutas hacia ubicaciones concretas relacionadas con los eventos.
- Desarrollar una solución accesible desde dispositivos móviles que garantice una experiencia de uso adecuada.
- Diseñar una arquitectura escalable que permita la incorporación de nuevas ciudades y eventos sin necesidad de rediseñar el sistema.

1.5. Estructura de la memoria

Se debe actualizar al terminar el TFG!!!!!!!!!!!!!!!!!!!!!!

La memoria se ha organizado en distintos capítulos en los cuales se puede ver la estructura y el desarrollo de dicho proyecto, de forma que se pueda comprender el proyecto de forma clara y completa.

En primer lugar, en este capítulo de introducción se presenta el contexto del proyecto, la motivación que lo impulsa, el estado actual de las distintas soluciones existentes, así como los objetivos que se llevan a cabo.

Después, en el capítulo de planificación se describe la metodología seguida durante el desarrollo de la aplicación, la organización del proyecto, además de las distintas fases de desarrollo y gestión de tareas.

Seguido por el capítulo de las iteraciones, en el cual se muestran las distintas etapas en las que se ha dividido y repartido el desarrollo de la aplicación, mostrando la evolución que ha ido teniendo el sistema a lo largo del desarrollo del proyecto.

Además, cuenta con un capítulo de estado, en el que se muestra el estado final del sistema desarrollado.

Por último, incluye un capítulo de pruebas en el que se analizan los resultados obtenidos al conseguir construir la versión final y operativa de la aplicación.

Esta memoria concluye con un capítulo de conclusiones, donde se valoran los objetivos conseguidos y se plantean posibles mejoras futuras.

Capítulo 2

Planificación

2.1. Metodología

Para el desarrollo de este proyecto se ha decidido hacer uso de una metodología **iterativa incremental** [?]. Este modelo se ha seleccionado porque permite organizar el proyecto en ciclos o iteraciones sucesivas, donde cada una añade funcionalidad al sistema de forma progresiva. Cada iteración incluye sus propias fases de análisis, diseño, implementación y pruebas, lo que facilita la validación continua del producto y la detección temprana de errores.

La elección de esta metodología está justificada por la naturaleza del proyecto, ya que permite entregar incrementos funcionales del sistema de manera gradual, comenzando por las funcionalidades básicas del módulo ciudadano y avanzando progresivamente hacia características más complejas como la geo-localización en tiempo real, los paneles de gestión y el sistema de notificaciones. Al ser un proyecto desarrollado por un único desarrollador, este modelo resulta adecuado porque combina la flexibilidad de adaptarse a posibles ajustes durante el desarrollo con una planificación estructurada que facilita el seguimiento del avance.

El proyecto se ha organizado en una **fase inicial de análisis** general, seguida de **cinco iteraciones** que abordan de forma incremental los distintos módulos del sistema, y una **fase final** de documentación, integración y entrega.

2.2. Planificación, Fases e iteraciones

2.2.1. Planificación inicial

Este proyecto consta de dos grandes fases, cuya planificación general se puede observar en el siguiente Diagrama de Gantt [?] realizado (Figura 2.1).

La **primera fase** se centra en el análisis de las aplicaciones relacionadas con la Semana Santa exis-

tentes, la identificación de sus carencias y áreas de mejora, y en la definición extendida de los requisitos funcionales y no funcionales [?] del sistema, además de la elaboración de los casos de uso [?] y los distintos diagramas que servirán como base para la realización del desarrollo de la aplicación.

En cuanto a la **segunda fase**, esta está más centrada en el diseño e implementación de la aplicación de forma iterativa e incremental [?]. Esta fase está organizada en cinco iteraciones principales. Aunque el marco general del proyecto sigue el modelo en cascada –en el sentido de que el análisis es seguido del diseño y tras este la implementación–, en esta segunda fase se aplica un ciclo interno de análisis, diseño, implementación y pruebas en cada iteración, lo que permite ir incorporando las funcionalidades de forma progresiva y verificando su correcto funcionamiento antes de darla por finalizada y avanzar a la siguiente. Estas iteraciones abarcan desde las funcionalidades básicas del ciudadano hasta las más complejas, como la geolocalización en tiempo real, es decir, la capacidad de conocer y actualizar la posición geográfica de una procesión de forma continua haciendo uso de la tecnología GPS¹ del móvil, o el sistema de notificaciones (mensajes enviados desde el servidor a los dispositivos de los usuarios sin que estos tengan que abrir la aplicación activamente).

Esta primera fase comienza con el análisis de las propuestas existentes en el mercado, identificando las características principales de aplicaciones como *El Penitente* [?], *Valladolid Cofrade* [?] o *sSantaZgz* [?].

Seguido de este análisis se realiza la definición de los requisitos funcionales, los cuales describen las funcionalidades concretas que el sistema debe ofrecer, y los requisitos no funcionales, que definen las condiciones de calidad, rendimiento y seguridad que debe cumplir [13], teniendo en cuenta los cuatro perfiles de usuario: Ciudadano, miembro de la organización, conjunto de organizaciones y Administrador. También se elaboran los diagramas de secuencia y el modelo de dominio, así como los casos de uso [14], representaciones estandarizadas de las interacciones entre los usuarios y el sistema, que servirán para el desarrollo posterior.

Las tareas de esta fase son:

- **Análisis de propuestas existentes** (8h): análisis de las aplicaciones actuales de Semana Santa y sus funcionalidades.
- **Definición de requisitos funcionales** (10h): identificación de las funcionalidades requeridas para cada perfil de usuario.
- **Definición de requisitos no funcionales** (5h): especificación de requisitos de rendimiento, seguridad, usabilidad y accesibilidad.
- **Diagramas de secuencia y clases** (8h): modelado de la estructura y comportamiento del sistema mediante notación UML [?].
- **Elaboración de casos de uso** (12h): descripción detallada de las interacciones usuario-sistema.

¹GPS (Global Positioning System): sistema de navegación por satélite que permite determinar la posición geográfica de un dispositivo en cualquier punto del planeta.

Planificación

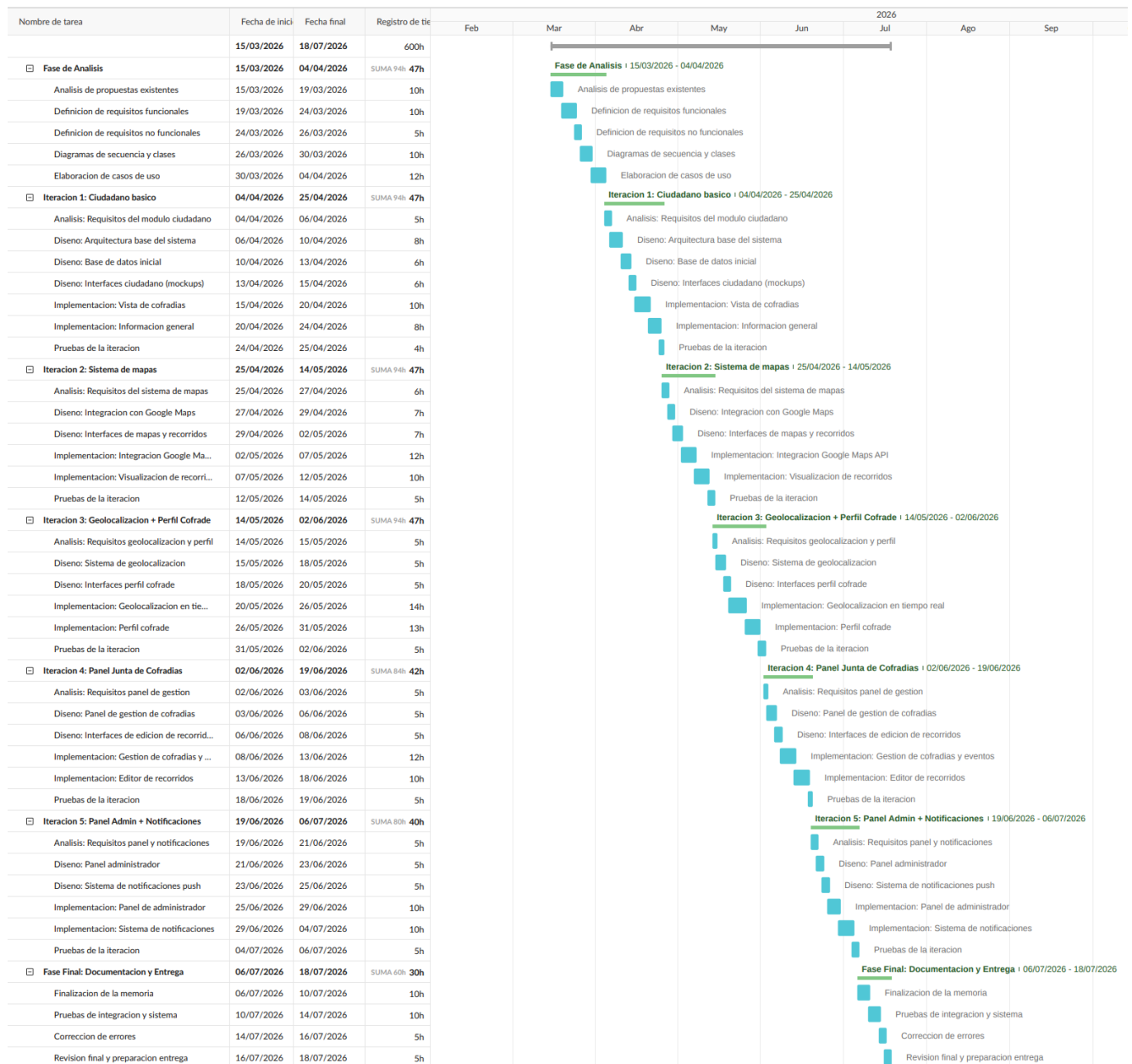


Figura 2.1: Diagrama de Gantt general completo del proyecto

2.2.2. Iteración 1: Módulo ciudadano básico

Esta primera iteración tiene como objetivo establecer la arquitectura base del sistema y desarrollar las funcionalidades fundamentales del perfil de ciudadano, entre las que se incluyen la visualización de la información general de la Semana Santa, el listado de organizaciones y la consulta de eventos.

- **Análisis:** requisitos específicos del módulo ciudadano (5h).
- **Diseño:** arquitectura base del sistema (8h), base de datos inicial (6h) e interfaces del ciudadano, incluyendo *mockups*² (6h).

²*Mockup*: representación visual estática de la interfaz de una aplicación, utilizada para planificar el diseño antes de su implementación.

- **Implementación:** vista de cofradías (10h) e información general (8h).
- **Pruebas:** validación de la iteración (4h).

2.2.3. Iteración 2: Sistema de mapas

Esta segunda iteración se centra principalmente en la implementación del sistema de mapas, que permitirá visualizar los distintos recorridos de las procesiones.

- **Análisis:** requisitos del sistema de mapas (4h).
- **Diseño:** integración con Google Maps [?], así como interfaces de mapas y recorridos (5h).
- **Implementación:** integración de la API de Google Maps [?] y visualización de recorridos (10h).
- **Pruebas:** validación de la iteración (4h).

2.2.4. Iteración 3: Geolocalización y perfil cofrade

Esta tercera iteración se centra en la implementación completa del sistema de geolocalización en tiempo real, de forma que se pueda conocer y actualizar de manera continua la posición de las procesiones durante su recorrido. Además, se desarrolla el perfil de miembro con sus correspondientes funcionalidades, incluyendo la posibilidad de compartir la ubicación durante las procesiones.

- **Análisis:** requisitos de geolocalización y perfil cofrade (4h).
- **Diseño:** sistema de geolocalización (5h), interfaces del perfil cofrade (5h).
- **Implementación:** geolocalización en tiempo real (14h), perfil cofrade (10h).
- **Pruebas:** validación de la iteración (4h).

2.2.5. Iteración 4: Panel de la Junta de Cofradías

Esta cuarta iteración corresponde al desarrollo del panel de gestión para las Juntas de Cofradías, permitiendo administrar la información de la Semana Santa, incluyendo eventos, recorridos, imágenes y participantes. El acceso a este panel está restringido mediante un mecanismo de control de acceso [?], que permite diferenciar los permisos y limitaciones de los distintos roles del sistema.

- **Análisis:** requisitos del panel de gestión (4h).
- **Diseño:** panel de gestión de cofradías (5h), interfaces de edición de recorridos (4h).
- **Implementación:** gestión de cofradías y eventos (12h), editor de recorridos (10h).
- **Pruebas:** validación de la iteración (4h).

2.2.6. Iteración 5: Panel de administrador y notificaciones

Esta quinta y última iteración se centra en la implementación del panel de administrador global del sistema, así como del sistema de notificaciones, que permitirá a la Junta de Cofradías comunicar a los usuarios cambios o incidencias en las procesiones de forma inmediata, sin necesidad de que estos tengan la aplicación abierta.

- **Análisis:** requisitos del panel de administrador y sistema de notificaciones (4h).
- **Diseño:** panel de administrador (4h), sistema de notificaciones *push* (4h).
- **Implementación:** panel de administrador (10h), sistema de notificaciones (10h).
- **Pruebas:** validación de la iteración (4h).

2.2.7. Fase final: documentación y entrega

En esta última fase se realizan las pruebas de integración del sistema completo, la corrección de errores detectados, la finalización de la memoria del proyecto y la preparación de la entrega final.

- **Finalización de la memoria** (10h).
- **Pruebas de integración y sistema** (8h).
- **Corrección de errores** (4h).
- **Revisión final y preparación de la entrega** (4h).

2.3. Análisis de riesgos

Otra parte importante en la planificación de un proyecto de desarrollo de software es el análisis de riesgos. Un riesgo es un evento de incertidumbre que puede afectar de forma positiva o negativa a los objetivos del proyecto [?]. Para garantizar el alcance de los objetivos del proyecto, es necesario realizar un análisis de estos riesgos.

Los riesgos identificados se organizan en función de su prioridad, la cual se determina mediante la técnica de análisis de exposición al riesgo (*Risk Exposure*, RE) [?], donde el riesgo se calcula mediante un valor numérico que representa la probabilidad e impacto estimados de cada evento. Se define de la siguiente manera:

$$RE = \text{probabilidad} \times \text{impacto} \quad (2.1)$$

La variable probabilidad representa la probabilidad de que el riesgo se materialice en el proyecto (escala de 1 a 10), mientras que el impacto representa la gravedad de sus consecuencias en caso de ocurrir (escala de 1 a 10). En la Tabla 2.1 se muestran los riesgos identificados en el proyecto.

Riesgo	Probabilidad	Impacto	Prioridad
1. Estimaciones de tiempo erróneas	8	7	56
2. Problemas con la integración de Google Maps	6	8	48
3. Errores durante el desarrollo	7	6	42
4. Falta de experiencia con tecnologías móviles	7	6	42
5. Problemas con la geolocalización en tiempo real	5	8	40
6. Cambios en los requisitos	6	6	36
7. Especificaciones ambiguas o poco definidas	5	6	30
8. Problemas con el entorno de desarrollo	4	5	20
9. Indisponibilidad del desarrollador	3	6	18
10. Problemas con Firebase o la base de datos	3	5	15

Tabla 2.1: Riesgos identificados en el proyecto

2.3.1. Planificación de riesgo

Tras realizar la identificación de los riesgos, el siguiente paso es la decisión de qué medidas se deben tomar para reducir la probabilidad de que el riesgo ocurra, de forma que se pueda disminuir su impacto una vez se materialice.

Riesgo	Estimaciones de tiempo erróneas
Reducción	Planificar las tareas con margen por si la estimación ha sido inferior al tiempo necesario. Dividir las tareas grandes en subtareas más pequeñas y manejables.
Contingencia	Cambiar la planificación, reducir el alcance del proyecto priorizando las funcionalidades esenciales.

Tabla 2.2: Riesgo 1: Estimaciones de tiempo erróneas

Riesgo	Problemas con la integración de Google Maps
Reducción	Realizar pruebas de concepto tempranas con la API de Google Maps. Documentarse sobre las limitaciones y cuotas del servicio.
Contingencia	Considerar alternativas como OpenStreetMap [?] o Mapbox en caso de problemas persistentes.

Tabla 2.3: Riesgo 2: Problemas con la integración de Google Maps

2.3.2. Monitorización de riesgos

Durante el desarrollo del proyecto, se debe realizar un seguimiento continuo de los riesgos identificados para detectar de forma temprana su aparición y poder aplicar las medidas pertinentes.

Riesgo	Errores durante el desarrollo
Reducción	Verificar que la funcionalidad está implementada correctamente antes de dar por finalizado el incremento. Realizar pruebas unitarias ^a .
Contingencia	Volver al último punto correctamente probado y continuar desde ahí. Utilizar control de versiones [?] para recuperar estados anteriores del código.

Tabla 2.4: Riesgo 3: Errores durante el desarrollo

^aPruebas unitarias: pruebas que verifican el correcto funcionamiento de componentes individuales del código de forma aislada.

Riesgo	Falta de experiencia con tecnologías móviles
Reducción	Formarse en las tecnologías clave antes de proceder al desarrollo. Consultar documentación oficial y tutoriales.
Contingencia	Aprovechar la experiencia del tutor para solventar los posibles problemas. Buscar soluciones en comunidades de desarrolladores.

Tabla 2.5: Riesgo 4: Falta de experiencia con tecnologías móviles

Riesgo	Problemas con la geolocalización en tiempo real
Reducción	Investigar las mejores prácticas para implementar geolocalización en aplicaciones móviles. Realizar prototipos tempranos para validar la viabilidad técnica.
Contingencia	Implementar un modo de actualización manual de la ubicación como alternativa temporal.

Tabla 2.6: Riesgo 5: Problemas con la geolocalización en tiempo real

Riesgo	Cambios en los requisitos
Reducción	El desarrollo iterativo permite adaptar el sistema a cambios de forma gradual.
Contingencia	Reevaluar las prioridades y ajustar el alcance del proyecto según las nuevas necesidades.

Tabla 2.7: Riesgo 6: Cambios en los requisitos

Riesgo	Especificaciones genéricas
Reducción	Reunir toda la información posible antes de abordar una tarea. Clarificar requisitos con el tutor.
Contingencia	Prototipar en caso de ser necesario para validar la interpretación de los requisitos.

Tabla 2.8: Riesgo 7: Especificaciones genéricas

!!Nota : Esta sección se completará al finalizar el desarrollo del proyecto, documentando qué riesgos se manifestaron, el impacto que tuvieron y las medidas que se tomaron para subsanarlos.

Riesgo	Problemas con el entorno de desarrollo
Reducción	Utilizar un repositorio remoto (GitHub [?]) para almacenar copias de seguridad del código.
Contingencia	Utilizar una máquina de respaldo o entorno de desarrollo alternativo.

Tabla 2.9: Riesgo 8: Problemas con el entorno de desarrollo

Riesgo	Indisposición del desarrollador
Reducción	Planificar las tareas con margen por si es necesario extender el tiempo para completar el incremento.
Contingencia	Realizar horas extra en caso necesario y ajustar la planificación.

Tabla 2.10: Riesgo 9: Indisposición del desarrollador

Riesgo	Problemas con Firebase/base de datos
Reducción	Documentarse sobre las limitaciones del plan gratuito de Firebase [?]. Diseñar una estructura de datos eficiente.
Contingencia	Migrar a un servicio alternativo en caso de problemas críticos.

Tabla 2.11: Riesgo 10: Problemas con Firebase/base de datos

2.4. Seguimiento del proyecto

Nota: Esta sección se completará al finalizar el desarrollo del proyecto, comparando la planificación inicial con la duración real del proyecto. comentar que ha habido distintas planificación debido a diversos cortatimepo que han surgido a lo largo de los mese de desarrollo

2.5. Estimación de costes

A continuación se presenta el presupuesto estimado del proyecto, calculado como si este fuese desarrollado por un equipo profesional completo, siguiendo los perfiles habituales en el desarrollo de una aplicación móvil de estas características. Se trata de una estimación teórica con fines de planificación y no de los costes reales asumidos por el autor, quien ha desarrollado el TFG de forma individual.

2.5.1. Costes de personal

Un proyecto de estas características, correspondiente al desarrollo completo de una aplicación móvil sobre la Semana Santa, requeriría en un entorno profesional un equipo multidisciplinar. Se han considerado los siguientes seis perfiles, entre los que se reparten las 300 horas estimadas de duración del proyecto:

- **Jefe de Proyecto:** coordinación general, planificación y seguimiento.
- **Analista Funcional:** recogida y especificación de requisitos, análisis del dominio.
- **Diseñador UX/UI:** diseño de la experiencia e interfaz de usuario.
- **Desarrollador Backend:** implementación del servidor, base de datos y API.
- **Desarrollador Frontend/Mobile:** implementación de la aplicación Android.
- **Tester/QA:** pruebas funcionales, de regresión y control de calidad.

Para cada perfil se ha calculado el coste por hora a partir del sueldo bruto anual medio en España, incrementado en un 30 % en concepto de cotizaciones a la Seguridad Social a cargo de la empresa, y dividido entre 1.800 horas anuales de jornada completa. La Tabla 2.12 recoge el reparto de horas y el coste asociado a cada perfil.

Perfil	Horas	Coste/hora	Coste total
Jefe de Proyecto [?]	45	33€	1.485,00€
Analista Funcional [?]	45	27€	1.215,00€
Diseñador UX/UI [?]	30	27€	810,00€
Desarrollador Backend [?]	75	26€	1.950,00€
Desarrollador Frontend/Mobile [?]	75	19€	1.425,00€
Tester/QA [?]	30	17€	510,00€
Total	300		7.395,00€

Tabla 2.12: Reparto de horas y coste de personal por perfil

2.5.2. Costes de equipo y amortización

El equipo del proyecto necesitaría un ordenador portátil por persona, además de un dispositivo Android dedicado a las pruebas. Se considera una duración total del proyecto de 5 meses.

- **Ordenadores portátiles:** 6 unidades valoradas en 1.000€ cada una, con una vida útil estimada de 5 años (60 meses). La amortización mensual por unidad es de 16,66€, lo que supone 83,30€ por portátil durante los 5 meses del proyecto, y **500,00€** en total para los 6 puestos de trabajo.
- **Dispositivo Android de pruebas:** 1 unidad valorada en 300€, con una vida útil estimada de 3 años (36 meses). La amortización mensual es de 8,33€, lo que supone **41,65€** para los 5 meses del proyecto.

El coste total de amortización de los recursos de equipo asciende a **541,65€**, tal y como recoge la Tabla 2.13.

Recurso	Valor	Vida útil	Amortización (5 meses)
Portátiles (x6)	6.000,00€	5 años	500,00€
Dispositivo Android (x1)	300,00€	3 años	41,65€
Total amortización			541,65€

Tabla 2.13: Amortización total de los recursos de equipo

2.5.3. Costes de licencias de software

Buena parte de las herramientas empleadas disponen de planes gratuitos o licencias universitarias suficientes para un proyecto de este tamaño. Sin embargo, un equipo profesional de 6 personas necesitaría además herramientas colaborativas de pago para la gestión del proyecto y el diseño. La Tabla 2.14 recoge el coste y la fuente de cada licencia.

Herramienta	Coste	Fuente
Android Studio	0,00€	[?]
Visual Studio Code	0,00€	[?]
Overleaf (plan gratuito)	0,00€	—
Google Firebase (plan gratuito)	0,00€	[?]
Google Maps API (crédito gratuito)	0,00€	[?]
GitHub (plan gratuito)	0,00€	[?]
Cuenta de desarrollador Google Play (pago único)	23,00€	[?]
Figma Professional (1 licencia, 5 meses)	69,00€	[?]
Jira Software Standard (6 usuarios, 5 meses)	218,40€	[?]
Total licencias	310,40€	

Tabla 2.14: Coste de licencias de software

2.5.4. Costes estructurales

Un equipo de 6 personas trabajando de forma presencial o híbrida necesitaría un espacio de oficina. Se ha tomado como referencia el alquiler de un despacho de 22 m² en un espacio de coworking en el centro de Valladolid, con todos los suministros incluidos (calefacción, refrigeración, electricidad, wifi y limpieza), por un importe de 475€ al mes [?]. Para los 5 meses de duración del proyecto, el coste estructural total asciende a **2.375,00€**.

2.5.5. Coste total

La Tabla 2.15 resume el presupuesto completo del proyecto, considerando personal, amortización de equipo, licencias de software y costes estructurales, para una duración de 300 horas repartidas en 5 meses.

Elemento	Coste
Personal (Tabla 2.12)	7.395,00€
Amortización de equipo (Tabla 2.13)	541,65€
Licencias de software (Tabla 2.14)	310,40€
Costes estructurales (oficina y suministros)	2.375,00€
Total	10.622,05€

Tabla 2.15: Costes totales del proyecto

Capítulo 3

Análisis

3.1. Requisitos

3.1.1. Requisitos funcionales

Los requisitos funcionales definen las diferentes funciones del sistema, además de cómo debe ser su comportamiento en ciertas situaciones. Cada requisito incluye, junto a su descripción, el nivel de prioridad asignado (Alta, Media o Baja), que indica su importancia relativa dentro del proyecto.

Para facilitar su lectura, los requisitos se han agrupado según el perfil de usuario (Sección ??) al que van dirigidos –ciudadano, cofrade, Junta de Cofradías o administrador–, y dentro de cada grupo se han ordenado de mayor a menor prioridad. Además el perfil de cofrade incluye, a mayores de los requisitos propios de dicho perfil, todos los requisitos del ciudadano, ya que un cofrade puede hacer uso de todas las funciones generales de la aplicación.

Ciudadano

- **RF-01** (*Prioridad: Alta*) El sistema debe permitir el uso de la aplicación sin la necesidad de realizar un registro previo.
- **RF-03** (*Prioridad: Alta*) El sistema debe mostrar la información detallada de las cofradías.
- **RF-04** (*Prioridad: Alta*) El sistema debe mostrar la información detallada de los eventos.
- **RF-05** (*Prioridad: Alta*) El sistema debe mostrar la información detallada de las procesiones.
- **RF-06** (*Prioridad: Alta*) El sistema debe permitir a los usuarios realizar búsquedas sobre eventos, procesiones o cofradías específicas.
- **RF-07** (*Prioridad: Alta*) El sistema debe permitir visualizar la posición de las procesiones en tiempo real mediante geolocalización.

- **RF-08** (*Prioridad: Alta*) El sistema debe poder generar rutas hacia la ubicación actual de una procesión.
- **RF-11** (*Prioridad: Alta*) El sistema debe permitir al usuario seleccionar la ciudad que desea consultar.
- **RF-13** (*Prioridad: Alta*) El sistema debe enviar notificaciones sobre los eventos o procesiones.
- **RF-14** (*Prioridad: Alta*) El sistema debe enviar alertas sobre los eventos o procesiones.
- **RF-17** (*Prioridad: Alta*) El sistema debe mostrar la información detallada de las procesiones (recorrido, horarios, pasos, ...).
- **RF-18** (*Prioridad: Alta*) El sistema debe solicitar permiso a los usuarios para acceder a su ubicación en tiempo real.
- **RF-09** (*Prioridad: Media*) El sistema debe poder generar rutas entre dos puntos evitando las procesiones activas.
- **RF-12** (*Prioridad: Media*) El sistema debe permitir a los usuarios marcar cofradías, eventos o procesiones como favoritas.
- **RF-16** (*Prioridad: Media*) El sistema debe permitir al usuario interactuar con el mapa (zoom, desplazamiento, ...).
- **RF-19** (*Prioridad: Media*) El sistema debe seleccionar automáticamente la ciudad más cercana en función de la ubicación del usuario.
- **RF-22** (*Prioridad: Media*) El sistema debe permitir al usuario configurar las preferencias de notificaciones.
- **RF-23** (*Prioridad: Media*) El sistema debe permitir filtrar cofradías, eventos y procesiones por distintos criterios (día, hora, tipo, etc.).
- **RF-10** (*Prioridad: Baja*) El sistema debe poder sugerir rutas hacia las ubicaciones más relevantes de una procesión (inicio, puntos de interés, fin, ...).
- **RF-24** (*Prioridad: Baja*) El sistema debe mostrar información sobre cortes de calles asociados a eventos y procesiones.
- **RF-25** (*Prioridad: Baja*) El sistema debe mostrar en el mapa puntos de interés cuando el usuario no haya seleccionado ninguna cofradía, evento o procesión.

Cofrade

- **RF-02** (*Prioridad: Alta*) El sistema debe permitir a los miembros de la organización acceder al modo de miembro.
- **RF-15** (*Prioridad: Alta*) El sistema debe permitir a los cofrades compartir su ubicación en tiempo real durante las procesiones.

Junta de Cofradías

- **RF-20** (*Prioridad: Alta*) El sistema debe permitir a la Junta de Cofradías editar información de cofradías, eventos y procesiones.
- **RF-21** (*Prioridad: Alta*) El sistema debe permitir a la Junta de Cofradías crear nuevas cofradías, eventos y procesiones.
- **RF-27** (*Prioridad: Alta*) El sistema debe permitir a la Junta de Cofradías crear alertas y notificaciones para los usuarios.
- **RF-28** (*Prioridad: Alta*) El sistema debe permitir a la Junta de Cofradías actualizar el estado de las procesiones en tiempo real.
- **RF-30** (*Prioridad: Alta*) El sistema debe permitir a la Junta de Cofradías definir y modificar los recorridos de las procesiones en el mapa.
- **RF-29** (*Prioridad: Media*) El sistema debe permitir a la Junta de Cofradías gestionar los puntos de interés de la ciudad.
- **RF-31** (*Prioridad: Media*) El sistema debe permitir a la Junta de Cofradías gestionar la información de los pasos de cada cofradía.
- **RF-32** (*Prioridad: Media*) El sistema debe permitir a la Junta de Cofradías publicar información sobre cortes de calles asociados procesiones.
- **RF-33** (*Prioridad: Media*) El sistema debe permitir a la Junta de Cofradías validar y aprobar el registro de nuevos cofrades en las cofradías, mediante el uso de un código.

Administrador

- **RF-26** (*Prioridad: Alta*) El sistema debe permitir al administrador gestionar las ciudades disponibles en la aplicación.

3.1.2. Requisitos de información

Los requisitos de información definen los datos que el sistema debe almacenar y gestionar para su correcto funcionamiento. Al igual que en los requisitos funcionales, se indica la prioridad de cada uno de ellos, ordenados de mayor a menor prioridad. En este caso no se agrupan por perfil de usuario, ya que describen el modelo de datos del sistema en su conjunto y no están asociados a un único actor.

- **RI-01** (*Prioridad: Alta*) El sistema debe almacenar la información sobre los miembros de las cofradías/hermandades.
- **RI-02** (*Prioridad: Alta*) El sistema debe almacenar la información sobre las cofradías/hermandades.

- **RI-03** (*Prioridad: Alta*) El sistema debe almacenar la información sobre los eventos.
- **RI-04** (*Prioridad: Alta*) El sistema debe almacenar la información sobre las procesiones.
- **RI-05** (*Prioridad: Alta*) El sistema debe almacenar la información sobre los recorridos.
- **RI-08** (*Prioridad: Alta*) El sistema debe almacenar la información sobre las alertas.
- **RI-09** (*Prioridad: Alta*) El sistema debe almacenar la información sobre las Juntas de Cofradías/hermandades.
- **RI-10** (*Prioridad: Alta*) El sistema debe almacenar la información sobre las ciudades.
- **RI-06** (*Prioridad: Media*) El sistema debe almacenar la información sobre las distintas ubicaciones que hay en el sistema.
- **RI-07** (*Prioridad: Media*) El sistema debe almacenar la información sobre los pasos.

3.1.3. Requisitos no funcionales

Los requisitos no funcionales describen las características de calidad del sistema, así como restricciones sobre su funcionamiento. También se indica la prioridad de cada uno de ellos, ordenados de mayor a menor prioridad. Al igual que los requisitos de información, no se agrupan por perfil de usuario, ya que son de aplicación transversal a todo el sistema.

- **RNF-01** (*Prioridad: Alta*) El sistema debe garantizar la seguridad de los datos de los usuarios, especialmente en lo relativo a la información de ubicación.
- **RNF-02** (*Prioridad: Alta*) La aplicación debe ser accesible y fácil de usar para usuarios sin experiencia previa.
- **RNF-05** (*Prioridad: Alta*) El sistema debe implementar mecanismos de control de acceso que diferencien entre usuarios normales, cofrades y miembros de la junta de cofrades.
- **RNF-07** (*Prioridad: Alta*) El sistema debe ser compatible tanto con dispositivos Android como con dispositivos iPhone.
- **RNF-08** (*Prioridad: Alta*) El sistema debe actualizar la posición de las procesiones con una frecuencia mínima de 30 segundos.
- **RNF-03** (*Prioridad: Media*) El sistema debe funcionar correctamente en dispositivos móviles con conexión a internet. Sin conexión, únicamente estará disponible la información estática de la aplicación, quedando inoperativas la geolocalización en tiempo real y las notificaciones.
- **RNF-04** (*Prioridad: Media*) El sistema debe gestionar de forma eficiente la carga del mapa, garantizando tiempos de respuesta menores a 10 segundos.
- **RNF-06** (*Prioridad: Media*) El sistema debe ser capaz de enviar notificaciones de manera fiable y en tiempo adecuado (menos de 60 segundos de latencia).

3.2. Casos de uso

Los casos de uso son los escenarios detallados que describen cómo interactúan los diferentes usuarios con la aplicación, desde la consulta de información hasta la gestión administrativa de la Semana Santa. Debido a que los usuarios pueden comportarse con diferentes roles (ciudadano, cofrade, Junta de Cofradías o administrador), es importante diferenciar los casos de uso entre cada uno de ellos. Para ello se han dividido los mismos en cuatro diagramas (ver figuras 3.1, 3.2, 3.3, 3.4), uno para cada rol.

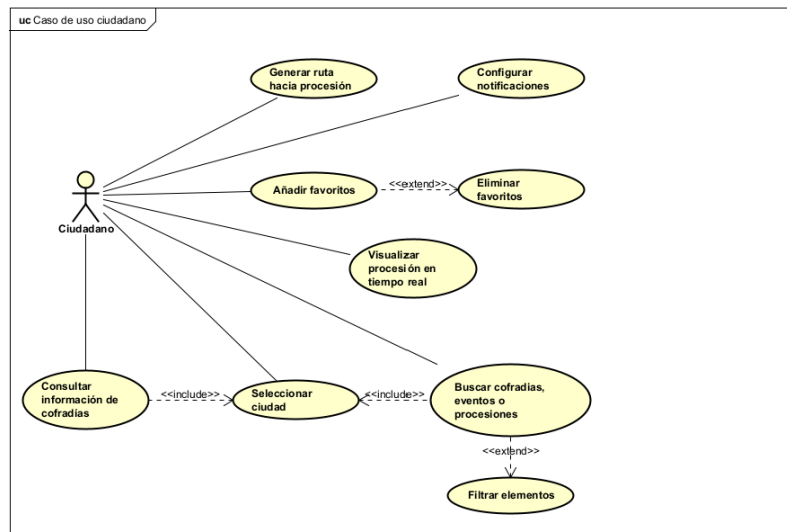


Figura 3.1: Diagrama de caso de uso Ciudadano

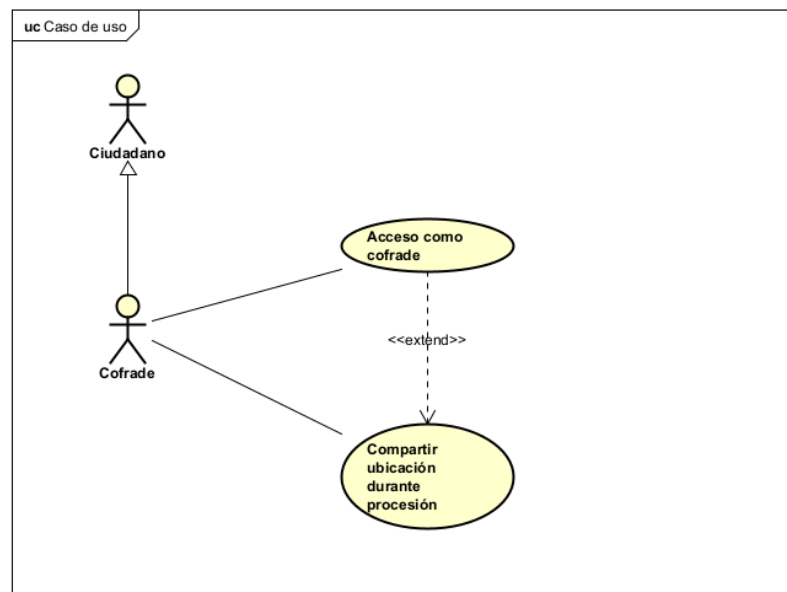


Figura 3.2: Diagrama de caso de uso Cofrade

En las siguientes tablas se exponen los cinco casos de uso más representativos del sistema, uno por cada rol implicado (ciudadano, cofrade, Junta de Cofradías y administrador), donde se detallan: el caso de uso, los actores que lo realizan, las precondiciones necesarias para iniciar el caso de uso, las postcondiciones que deben cumplirse al completarse, una secuencia base de pasos, secuencias alternativas que

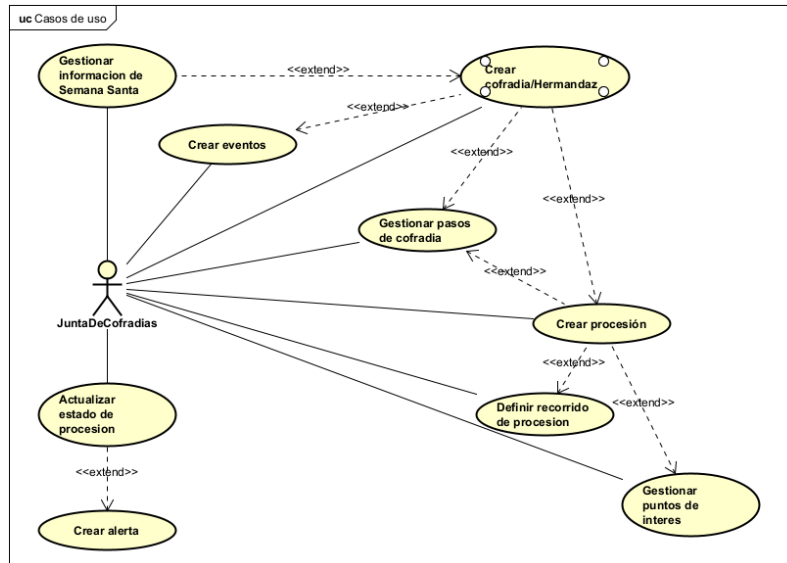


Figura 3.3: Diagrama de caso de uso Junta de Cofradías

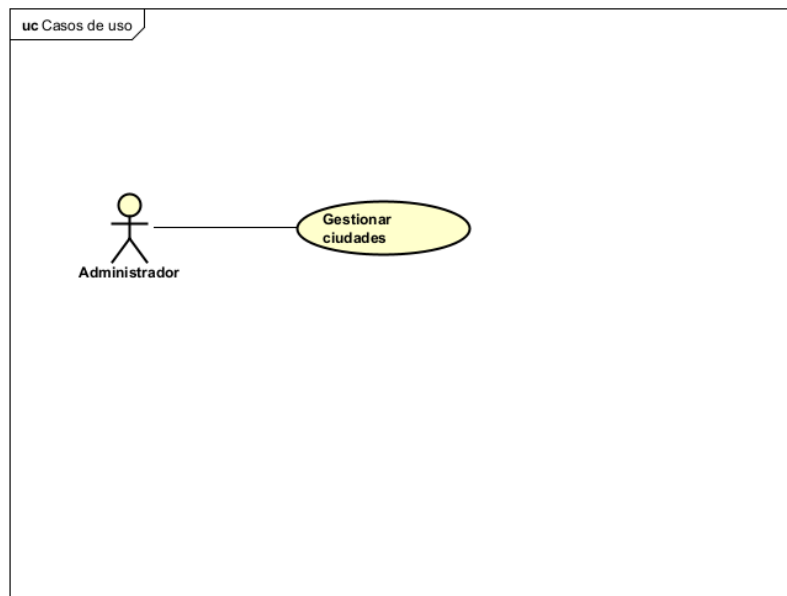


Figura 3.4: Diagrama de caso de uso Administrador

pueden darse durante la ejecución y los requisitos relacionados con el caso de uso. El resto de los casos de uso identificados, de carácter más secundario, se recogen en el Apéndice ??.

3.3. Modelo de dominio

En esta sección se detalla el diseño conceptual del sistema, incluyendo las entidades clave y sus relaciones. El modelo refleja la estructura lógica del dominio de la Semana Santa, garantizando una representación fiel de sus procesos, actores y reglas de negocio. Se explican brevemente las abstracciones incluidas en la Figura 4.3.

Caso de uso: Visualizar procesión en tiempo real	
Resumen	El usuario visualiza la posición actual de una procesión en el mapa.
Actor	Ciudadano
Precondición	La procesión está en curso El usuario tiene conexión a internet
Postcondición	Se muestra la lista de cofradías con su información.
Secuencia Base	
[1] El usuario selecciona una procesión activa.	
[2] El sistema muestra el mapa con el recorrido de la procesión.	
[3] El sistema actualiza la posición en tiempo real.	
[4] El usuario puede interactuar con el mapa (zoom, desplazamiento).	
Secuencia Alternativa	
[1.a.1] La procesión no está activa: El sistema muestra el recorrido planificado sin posición en tiempo real.	
[3.a.1] Pérdida de conexión: El sistema muestra la última posición conocida.	
Requisitos Relacionados	RF-05, RF-14 y RF-16.

Tabla 3.1: Caso de uso: Visualizar procesión en tiempo real

Caso de uso: Compartir ubicación durante procesión	
Resumen	Un cofrade comparte su ubicación en tiempo real durante una procesión.
Actor	Cofrade
Precondición	El cofrade está registrado, el cofrade tiene permisos de ubicación y hay una procesión de su cofradía en curso
Postcondición	La ubicación del cofrade se transmite al sistema.
Secuencia Base	
[1] El cofrade accede a la procesión activa de su cofradía.	
[2] El cofrade activa la opción de compartir ubicación.	
[3] El sistema solicita permisos de ubicación si no los tiene.	
[4] El cofrade concede los permisos.	
[5] El sistema comienza a transmitir la ubicación.	
[6] El sistema muestra indicador de ubicación compartida.	
Secuencia Alternativa	
[4.a.1] El usuario deniega permisos: El sistema informa que no puede compartir ubicación sin permiso.	
[2.a.1] Cancelación de la operación: El usuario desea cancelar la operacion.	
[5.a.1] Pérdida de conexión: El sistema muestra la última posición conocida.	
Requisitos Relacionados	RF-13 y RF-17.

Tabla 3.2: Caso de uso: Compartir ubicación durante procesión

Caso de uso: Crear alerta	
Resumen	La Junta de Cofradías crea una alerta para informar a los usuarios sobre incidencias o información relevante.
Actor	Junta de Cofradías
Precondición	El usuario tiene rol de Junta de Cofradías.
Postcondición	La alerta queda creada y se envían notificaciones a los usuarios afectados.
Secuencia Base	
[1] El usuario accede al panel de gestión de alertas.	
[2] El usuario selecciona crear nueva alerta.	
[3] El sistema muestra el formulario de creación.	
[4] El usuario selecciona el tipo de alerta.	
[5] El usuario introduce título y mensaje de la alerta.	
[6] El usuario selecciona la prioridad de la alerta.	
[7] El usuario define fecha de expiración (opcional).	
[8] El usuario selecciona los destinatarios.	
[9] El usuario confirma la creación.	
[10] El sistema crea la alerta y envía notificaciones.	
Secuencia Alternativa	
[4.a.1] Cancelación de la operación: El usuario desea cancelar la operación.	
[5.a.1] Cancelación de la operación: El usuario desea cancelar la operación.	
[6.a.1] Cancelación de la operación: El usuario desea cancelar la operación.	
[7.a.1] Cancelación de la operación: El usuario desea cancelar la operación.	
[8.a.1] Cancelación de la operación: El usuario desea cancelar la operación.	
[9.a.1] Datos incompletos: El sistema muestra los campos requeridos.	
[10.a.1] Error al enviar notificaciones: El sistema reintenta y registra el error.	
Requisitos Relacionados	RF-11, RF-12 y RF-26.

Tabla 3.3: Caso de uso: Crear alerta

Caso de uso: Generar ruta hacia procesión	
Resumen	El usuario genera una ruta desde su ubicación hasta una procesión.
Actor	Ciudadano
Precondición	El usuario tiene permisos de ubicación y hay una procesión seleccionada
Postcondición	Se muestra la ruta en el mapa
Secuencia Base	
[1] El usuario selecciona una procesión.	
[2] El usuario pulsa generar ruta.	
[3] El sistema obtiene la ubicación del usuario.	
[4] El sistema calcula la ruta óptima.	
[5] El sistema muestra la ruta en el mapa.	
Secuencia Alternativa	
[3.a.1] Sin permisos de ubicación: El sistema solicita permisos.	
[4.a.1] Sin conexión: El sistema muestra error de conexión.	
[5.a.1] Cancelación de la operación: El usuario desea cancelar la operacion.	
Requisitos Relacionados	RF-07 y RF-17.

Tabla 3.4: Caso de uso: Generar ruta hacia procesión

Caso de uso: Gestionar ciudades disponibles	
Resumen	El administrador da de alta, edita o retira ciudades disponibles en la aplicación.
Actor	Administrador
Precondición	El usuario tiene rol de Administrador.
Postcondición	La lista de ciudades disponibles queda actualizada en el sistema.
Secuencia Base	
[1] El usuario accede al panel de administración.	
[2] El sistema muestra la lista de ciudades existentes.	
[3] El usuario selecciona añadir, editar o eliminar una ciudad.	
[4] El usuario introduce o modifica los datos de la ciudad.	
[5] El sistema valida y guarda los cambios.	
Secuencia Alternativa	
[4.a.1] Cancelación de la operación: El usuario desea cancelar la operación.	
[5.a.1] Datos incompletos: El sistema muestra los campos requeridos.	
[3.a.1] Ciudad con datos asociados: El sistema advierte antes de eliminar una ciudad que tenga cofradías, eventos o procesiones registrados.	
Requisitos Relacionados	RF-26.

Tabla 3.5: Caso de uso: Gestionar ciudades disponibles

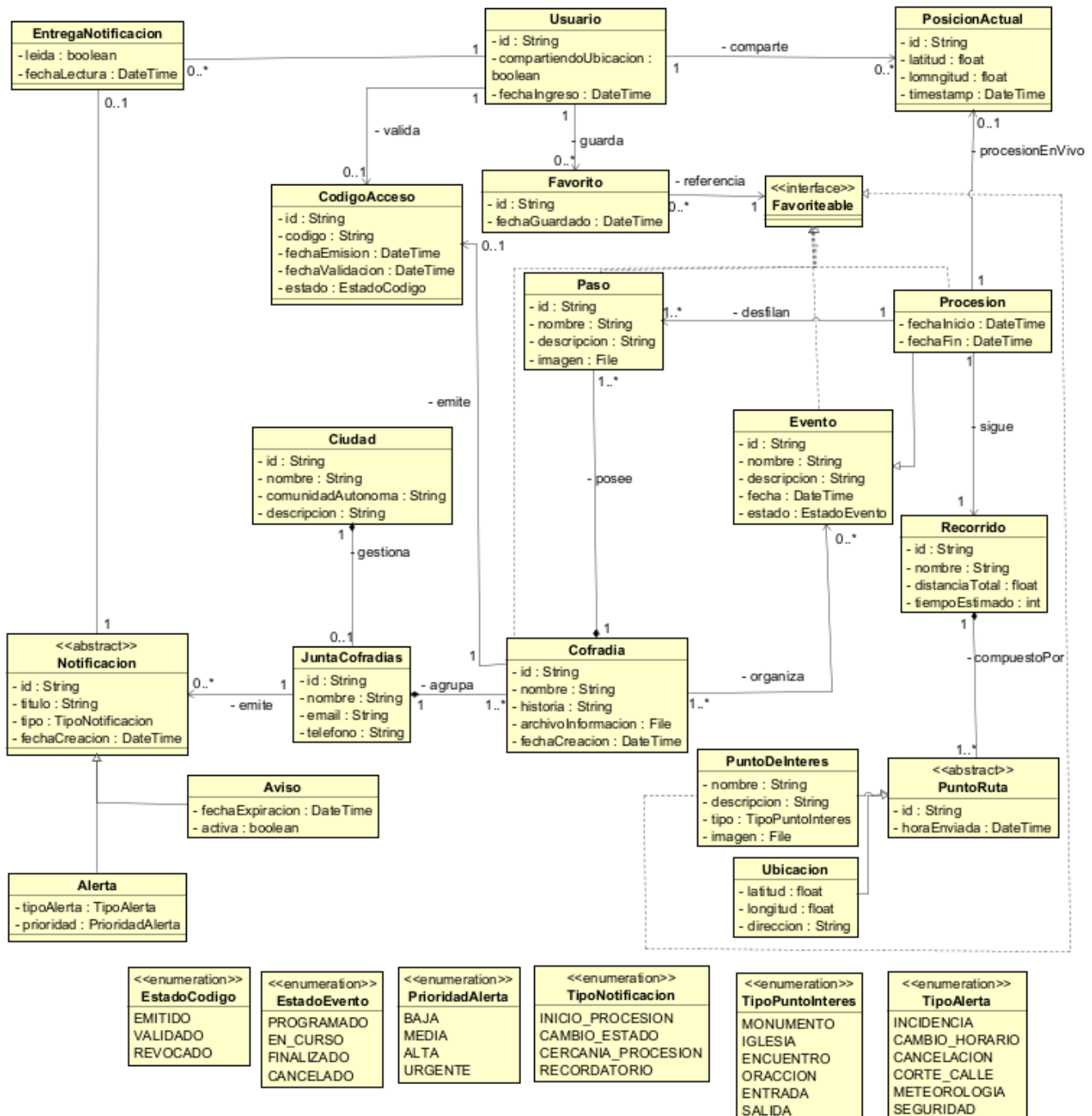


Figura 3.5: Diagrama de dominio del proyecto

Capítulo 4

Descripción de las iteraciones

4.1. 1ª Iteración

4.1.1. Análisis

4.1.2. Diseño

Antes de comenzar la implementación de esta primera iteración ha sido necesario tomar una serie de decisiones de diseño que condicionan el resto del proyecto: qué tecnologías se van a usar, cómo se organiza el código, cómo se guardan los datos y qué soluciones ya conocidas (patrones de diseño) conviene aplicar. Estas decisiones se explican a continuación con el detalle suficiente para que puedan entenderse sin conocimientos previos de React Native o Firebase, ya que sientan la base para el resto de iteraciones del proyecto.

Decisiones tecnológicas

Para el cliente¹ móvil se ha optado por **React Native** [?], un *framework*² que permite escribir una única aplicación y ejecutarla tanto en Android como en iOS, en lugar de programar dos aplicaciones nativas independientes (una en Kotlin/Java para Android y otra en Swift para iOS). Se descarta el desarrollo nativo duplicado por dos motivos: el requisito RNF-07 exige que la aplicación funcione en ambas plataformas, y al tratarse de un proyecto desarrollado por una única persona, mantener y probar dos bases de código distintas multiplicaría el tiempo de desarrollo sin aportar ninguna ventaja funcional. Además, React Native dispone de librerías ya probadas para los dos puntos más críticos del sistema: la integración con mapas (`react-native-maps`) y el acceso a la geolocalización del dispositivo (`expo-location` o `react-native-geolocation-service`).

¹Cliente: en una arquitectura cliente-servidor, es la parte del sistema con la que interactúa directamente el usuario; en este caso, la propia aplicación instalada en el teléfono móvil.

²Framework: conjunto de herramientas y bibliotecas de código que facilitan el desarrollo de un tipo concreto de aplicación, proporcionando una estructura base sobre la que trabajar.

Como *backend*³ se ha optado por **Firestore** [?], una plataforma de Google que se ofrece como *Backend-as-a-Service* (BaaS): en lugar de programar un servidor propio, se contratan una serie de servicios ya contruidos (base de datos, autenticación de usuarios, almacenamiento de ficheros, envío de notificaciones) a los que la aplicación se conecta directamente. Se descarta programar un backend a medida porque Firestore ya resuelve de forma nativa todo lo anterior y, sobre todo, porque su base de datos (Cloud Firestore [?]) permite que varios dispositivos vean los mismos datos actualizados en tiempo real sin que el equipo de desarrollo tenga que programar esa sincronización a mano: la aplicación simplemente se “suscribe” a los datos que le interesan y Firestore le avisa automáticamente cada vez que cambian (este mecanismo se denomina *listener*, y se explica con más detalle en el apartado de patrones de diseño). Esta decisión es coherente con el riesgo “Problemas con Firestore/base de datos” identificado en la planificación (Sección 2.3) y con el requisito RNF-08, que exige actualizar la posición de una procesión cada 30 segundos como mínimo.

Para los mapas se mantiene **Google Maps Platform** [?], ya justificado en el estado de la cuestión (Sección 1.3.2), por ser la solución de mapas más extendida y con mejor documentación. La contingencia prevista ante el riesgo “Problemas con la integración de Google Maps” (Sección 2.3) sigue siendo migrar a OpenStreetMap [?] o Mapbox si en la práctica surgen problemas de cuota o de coste.

Arquitectura del sistema

La arquitectura de un sistema describe, a grandes rasgos, en qué piezas se divide y cómo se comunican entre sí. En este proyecto se ha definido una arquitectura **cliente-servidor sin servidor intermedio propio**: la aplicación React Native (el cliente) se comunica directamente con los SDK⁴ oficiales de Firestore (Authentication, Cloud Firestore, Cloud Messaging y Storage) y de Google Maps Platform (Maps SDK y Directions API). Es importante notar que no existe una capa de API REST⁵ propia entre el cliente y el servidor, como sí ocurriría si se hubiese optado por programar un backend a medida: es Firestore quien ya expone esa comunicación a través de su SDK.

Estructura del front-end Dentro del propio cliente, el código se organiza siguiendo una **arquitectura en capas** [?]: el proyecto se divide en varios grupos de carpetas (o *paquetes*), cada uno con una responsabilidad concreta, de forma que cada capa solo puede depender de la capa inmediatamente inferior y nunca al revés. Esta regla se sigue para que un cambio en una capa concreta (por ejemplo, cambiar de Firestore a otro proveedor de datos) afecte al menor número posible de partes del código, y para poder probar cada capa de forma aislada. Las cuatro capas definidas son:

- **UI** (screens, components, navigation): las pantallas que ve el usuario, agrupadas por su rol (ciu-

³Backend: la parte del sistema que no ve el usuario directamente, encargada de almacenar los datos, gestionar la lógica de negocio y responder a las peticiones que le hace el cliente.

⁴SDK (*Software Development Kit*): conjunto de herramientas que proporciona un servicio (en este caso, Firestore o Google Maps) para que otras aplicaciones puedan integrarse con él fácilmente, sin tener que hablar directamente con sus servidores a bajo nivel.

⁵API REST: forma habitual de comunicación entre un cliente y un servidor propio, en la que el cliente pide o envía datos a través de peticiones HTTP a unas direcciones (*endpoints*) definidas por el propio equipo de desarrollo. Al no tener servidor propio, esta capa no es necesaria en este proyecto.

dadano, cofrade, junta de cofradías, administrador), siguiendo los cuatro diagramas de casos de uso del Capítulo 3; los componentes visuales reutilizables entre pantallas (por ejemplo, la tarjeta con la información de una cofradía); y la navegación entre pantallas.

- **Application** (context, hooks): el estado que se comparte entre varias pantallas a la vez, como la sesión del usuario, su rol activo, la ciudad seleccionada o si está compartiendo su ubicación en ese momento.
- **Data** (services, models): un servicio por cada tipo de información del dominio (cofradiaService, procesionService, geoService, notificationService...) que es el único responsable de hablar con Firebase y con Google Maps, además de los tipos de datos que describen cómo es cada objeto (una cofradía, una procesión...) dentro de la aplicación.
- **Infrastructure** (firebase, maps): la configuración e inicialización de los SDK externos, es decir, el código mínimo necesario para que el resto de la aplicación pueda empezar a usar Firebase y Google Maps.

La Figura 4.1 muestra esta organización en forma de diagrama de paquetes, donde las flechas discontinuas indican qué paquete depende de (necesita) cuál.

Estructura del back-end Como se ha explicado, el back-end de este proyecto no incluye código propio, sino que se apoya íntegramente en los siguientes servicios de Firebase:

- **Firebase Authentication:** servicio que crea y gestiona la sesión de los usuarios con un rol elevado (cofrade, junta de cofradías, administrador), sin que el proyecto tenga que programar ni almacenar contraseñas por su cuenta. El ciudadano no necesita registrarse ni iniciar sesión en ningún momento: puede consultar procesiones, ver el mapa y recibir notificaciones sin tener ningún tipo de cuenta. Solo cuando un usuario introduce un código de acceso válido (colección `codigosAcceso`, ver apartado Diseño de la base de datos) se crea su sesión de Firebase Authentication y se le asigna el rol correspondiente.
- **Cloud Firestore:** la base de datos del proyecto. Guarda toda la información en las colecciones `ciudades`, `cofradias`, `eventos`, `procesiones`, `recorridos`, `usuarios` y `notificaciones` (el diseño completo de estas colecciones se explica en el apartado Diseño de la base de datos de esta misma sección).
- **Firebase Storage:** espacio de almacenamiento en la nube donde se guardan las imágenes de los pasos de cada cofradía, ya que este tipo de archivo no se guarda de forma eficiente dentro de la base de datos.
- **Cloud Messaging** [?]: servicio que permite enviar notificaciones *push* (avisos que llegan al teléfono del usuario aunque no tenga la aplicación abierta en ese momento), usado para las alertas y avisos que publican las Juntas de Cofradías (RF-13, RF-14, RF-27).

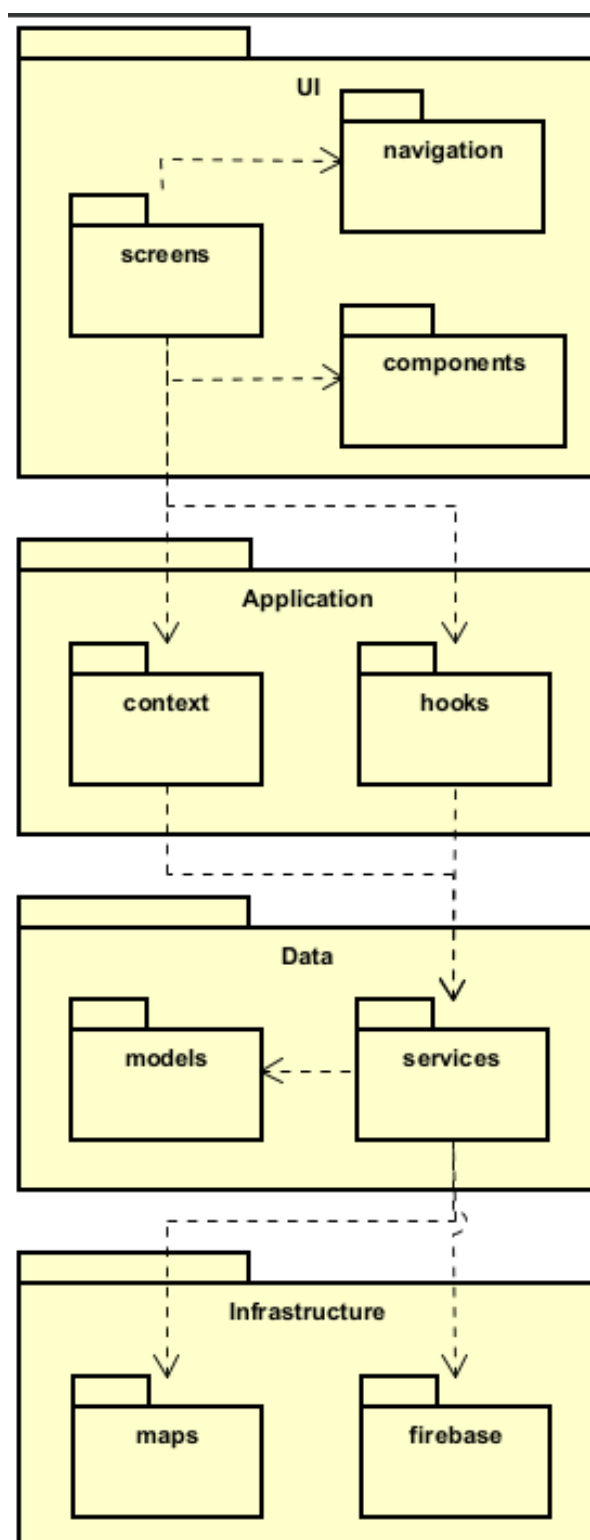


Figura 4.1: Diagrama de paquetes de la aplicación cliente

Diagrama de despliegue Un diagrama de despliegue representa, de forma física, dónde se ejecuta cada parte del sistema y cómo se comunican entre sí esos entornos. La Figura 4.2 recoge el despliegue completo de este proyecto: el dispositivo móvil ejecuta la aplicación React Native, que se comunica mediante el

protocolo HTTPS⁶ tanto con Firebase (autenticación, base de datos, almacenamiento y notificaciones) como con Google Maps Platform (visualización de mapas y cálculo de rutas). Puede observarse que no existe ningún nodo de base de datos local: el requisito RNF-03 (disponibilidad de la información estática sin conexión a internet) queda cubierto por la caché⁷ que Cloud Firestore guarda automáticamente en el dispositivo, sin que sea necesario implementar una base de datos local adicional como haría, por ejemplo, una aplicación Android nativa con Room.

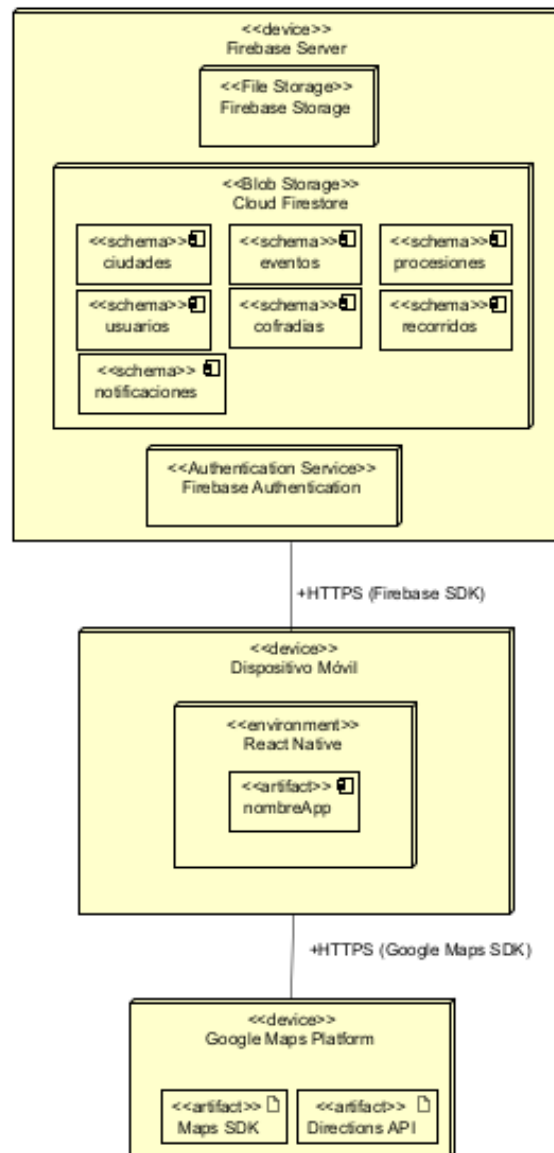


Figura 4.2: Diagrama de despliegue de la aplicación

⁶HTTPS: versión segura (cifrada) del protocolo HTTP, usado habitualmente para la comunicación entre aplicaciones y servicios en internet.

⁷Caché: copia local y temporal de unos datos, guardada en el propio dispositivo, que permite seguir accediendo a ellos aunque se pierda la conexión a internet.

Diseño de la base de datos

A diferencia de una base de datos relacional tradicional (como MySQL), organizada en tablas con filas y columnas relacionadas mediante claves, Cloud Firestore es una base de datos **NoSQL** [?]⁸ de tipo documental: la información se organiza en *colecciones* (agrupaciones de elementos del mismo tipo, similar a una carpeta) que contienen *documentos* (cada elemento individual, similar a un fichero, con sus propios campos de datos). El modelo de dominio de la Figura 4.3 se ha traducido a esta estructura de colecciones y subcolecciones (colecciones anidadas dentro de un documento concreto) siguiendo estas decisiones:

- `pasos` y `puntosRuta` se modelan como **subcolecciones** de `cofradías` y `recorridos` respectivamente, porque siempre se consultan junto a su documento padre (por ejemplo, los pasos de una cofradía concreta) y nunca de forma independiente.
- `posicionActual` es un **documento único** dentro de cada procesión, que se sobrescribe cada vez que llega una nueva posición GPS. El cliente se suscribe a este documento mediante un *listener* en tiempo real (en vez de estar preguntando repetidamente si hay novedades, es Firestore quien avisa a la aplicación en cuanto el documento cambia), cumpliendo el requisito RNF-08 sin necesidad de implementar servidores de sockets propios.
- `favoritos` no se guarda en Firestore, sino de forma local en el propio dispositivo (por ejemplo, con `AsyncStorage`), ya que el ciudadano no tiene cuenta ni sesión en la que persistir esta información en el servidor; únicamente se guarda el identificador de la cofradía o procesión marcada como favorita.
- `codigosAcceso` y `entregas`, estas sí como subcolecciones de Firestore, guardan únicamente una **referencia** (el identificador) a la cofradía, procesión o usuario correspondiente, en lugar de copiar todos sus datos, para evitar tener la misma información duplicada en varios sitios y que se desactualice.
- La colección `usuarios` solo contiene los perfiles de rol elevado (cofrade, junta de cofradías, administrador): el ciudadano no se registra, por lo que no genera ningún documento en Firestore para poder usar la aplicación. Un usuario obtiene uno de estos roles al introducir un **código de acceso** válido, gestionado en la colección `codigosAcceso` (con estados `EMITIDO`, `VALIDADO` y `REVOCADO`); en ese momento se crea su sesión de Firebase Authentication y su documento en `/usuarios/{id}`, con el rol guardado como campo y comprobado mediante las reglas de seguridad de Firestore, mecanismo que cubre el requisito RNF-05 (control de acceso diferenciado por rol). Cada código de acceso lleva además una **referencia a la cofradía** que lo emite, de forma que un cofrade que valida su código queda automáticamente vinculado a esa cofradía y puede actuar en su nombre (por ejemplo, compartiendo la ubicación de uno de sus pasos o publicando un evento).

⁸NoSQL: familia de bases de datos que no organiza la información en tablas relacionadas, sino en otras estructuras –en este caso, documentos agrupados en colecciones– más flexibles para datos que cambian de forma o que se leen con mucha frecuencia en tiempo real.

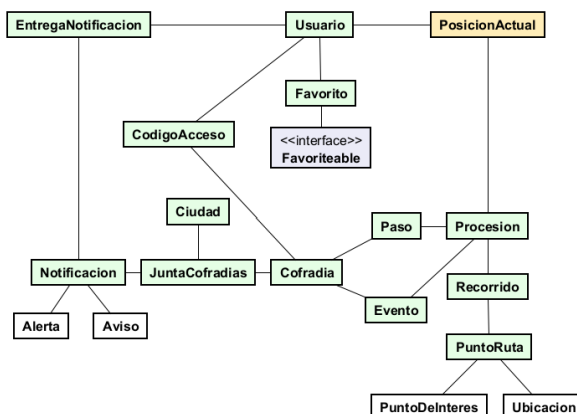


Figura 4.3: Diagrama de dominio

Patrones de diseño

Un patrón de diseño [?] es una solución ya conocida y reutilizable a un problema que se repite con frecuencia en el desarrollo de software; su uso facilita la legibilidad, el mantenimiento y la escalabilidad del código. Para seleccionar los patrones aplicables a este proyecto se han revisado dos TFG previos de la escuela con una arquitectura similar (aplicación móvil apoyada en Firebase), de los que se han descartado los patrones *Builder* y *Abstract Factory*: en Java se justifican porque los constructores de clases no admiten bien parámetros opcionales, pero en JavaScript/TypeScript un objeto literal con esos mismos campos opcionales ya resuelve el mismo problema de forma nativa, y en ningún punto del sistema es necesario intercambiar familias completas de implementaciones. Los siete patrones finalmente aplicados son los siguientes:

- **Singleton:** patrón que garantiza que una clase tenga una única instancia en toda la aplicación y un punto de acceso global a ella, evitando crear varias copias innecesarias. En este proyecto, la inicialización del SDK de Firebase (`firebase/config`) se realiza una única vez y esa misma instancia se reutiliza en el resto de la aplicación.
- **Repository:** patrón que separa la lógica de acceso a los datos del resto de la aplicación, ocultando los detalles de cómo y dónde se almacenan. La capa `services` es la única parte del código que conoce el SDK de Firebase; el resto de la aplicación solo invoca funciones como `procesionService.obtenerActividad` sin saber que por detrás se está consultando Firestore. Esto facilitaría cambiar de proveedor de backend en el futuro (línea de trabajo futuro, Sección 7.1) sin modificar ninguna pantalla.
- **Observer:** patrón en el que uno o varios elementos (“observadores”) se suscriben a los cambios de otro elemento y son notificados automáticamente cuando este cambia, en lugar de tener que preguntar repetidamente. Los *listeners* de Firestore (`onSnapshot`) funcionan así: notifican a la interfaz cada vez que cambia la posición de una procesión o llega una notificación nueva.
- **Factory Method:** patrón que centraliza en una única función la creación de un objeto, delegando en ella la decisión de qué variante concreta construir. El dominio distingue Aviso y Alerta como subtipos de Notificacion (Figura 4.3); una función fábrica construye el objeto correcto según los campos `tipoAlerta` y `fechaExpiracion` antes de guardarlo en Firestore.

- **Context/Provider:** convención propia de React (no un patrón GoF⁹ clásico) que permite exponer un dato o estado a cualquier pantalla de la aplicación sin tener que pasarlo manualmente de componente en componente. `AuthContext` y `LocationContext` exponen así el usuario autenticado y los permisos de ubicación a cualquier pantalla que los necesite.
- **Facade:** patrón que ofrece una única función sencilla como punto de entrada a una operación que, por detrás, involucra varios pasos o subsistemas distintos. El caso de uso “Compartir ubicación durante procesión” (RF-15) esconde tras la llamada `geoService.iniciarCompartirUbicacion(procesionId)` la solicitud de permiso de GPS, la escritura periódica en Firestore y la gestión de una posible pérdida de conexión.
- **Adapter:** patrón que convierte el formato de datos de un sistema externo al formato interno que espera la aplicación, permitiendo que ambos “encajen” sin modificar ninguno de los dos. Las notificaciones *push* de Firebase Cloud Messaging llegan con un formato plano (título, cuerpo, y un mapa de texto), distinto del modelo interno tipado `Alerta/Aviso`; una función adaptadora convierte ese *payload* recibido al modelo de dominio antes de utilizarlo en la aplicación.

4.1.3. Implementación

4.1.4. Pruebas

4.2. 2ª Iteración

4.3. 3ª Iteración

4.4. 4ª Iteración

⁹GoF (*Gang of Four*): apodo con el que se conoce a los cuatro autores del libro *Design Patterns* [?] (1994), que popularizó el catálogo clásico de patrones de diseño orientado a objetos (Singleton, Observer, Factory Method, entre otros).

Capítulo 5

Estado final de la aplicación

5.0.1. Historias de usuario

ID	HU01
Nombre	Login de un usuario
Prioridad	Alta
Riesgo	Bajo
Descripción	Como usuario quiero poder acceder a la aplicación a través de una pantalla de login
Validación	- Quiero poder introducir mi nick y contraseña para acceder a mi perfil de usuario - Quiero que mi nick sea único

Tabla 5.1: Historia de usuario HU01

Capítulo 6

Pruebas

Capítulo 7

Conclusiones

7.1. Trabajo futuro

Como línea de trabajo futuro, se plantea la posibilidad de generalizar el sistema desarrollado para dar soporte no solo a la Semana Santa, sino a cualquier tipo de evento o celebración tradicional (por ejemplo, ferias, romerías o fiestas patronales). Para ello, entre otros cambios, las entidades específicas del dominio actual deberían cambiarse por nombres de clases más genéricas: las cofradías o hermandades pasarían a modelarse como *organizaciones*, sus miembros o cofrades como *miembros*, y las Juntas de Cofradías como *conjuntos de organizaciones* encargados de coordinar a varias organizaciones dentro de un mismo evento. De esta forma, el resto del sistema (geolocalización en tiempo real, gestión de recorridos, notificaciones, etc.) podría reutilizarse prácticamente sin cambios, ya que estas funcionalidades no dependen de la naturaleza religiosa del evento, sino de su estructura organizativa. Esta generalización estaría alineada con el objetivo de escalabilidad planteado inicialmente para el proyecto (Sección 1.4), y permitiría ampliar el alcance de la aplicación mucho más allá del ámbito de la Semana Santa.

Apéndices

Apéndice A

Acrónimos

- **API:** Application Programming Interface.
- **PWA:** Progressive Web App (ya lo explicas en el texto, pero debería estar en el apéndice)
- **GPS:** Global Positioning System
- **UML:** Unified Modeling Language
- **RF:** Requisito Funcional
- **RI:** Requisito de Información
- **RNF:** Requisito No Funcional
- **HU:** Historia de Usuario
- **TFG:** Trabajo de Fin de Grado

Apéndice B

Manual de despliegue

Apéndice C

Manual de uso

Apéndice D

Contenido del TFG

